

Holiday Proximity Feature Engineering

ROSSMANN STORE SALES FORECASTING COMPETITION HANDBOOK

In retail sales forecasting, the historical calendar behaves non-linearly. Sales volume doesn't simply spike on an individual public holiday; instead, profound demand patterns manifest in the surrounding days. Customers routinely engaging in "panic buying" cause massive surges immediately prior to closures, followed by steep drop-offs when stores reopen.

Because gradient-boosted tree architectures (such as XGBoost, LightGBM, and CatBoost) struggle to naturally deduce chronological sequence metrics relative to moving milestones, you must explicitly engineer distance vectors. Below are three premier methodologies for encoding holiday proximity constraints.

Approach 1: Continuous Timedelta Displacements ("Days Since / Days Until")

This is the standard and highly effective approach. For every active daily record in the dataset, you compute continuous integers mapping the precise time difference to the immediate past holiday and the upcoming prospective holiday. Grouping operations by individual stores ensures tracking continuity without data leakage.

```
import pandas as pd
import numpy as np

# Ensure DataFrame is sorted chronologically per retail branch
df = df.sort_values(['Store', 'Date']).reset_index(drop=True)

def calculate_holiday_distances(df):
    # Materialize isolated event tracking column
    df['HolidayDate'] = np.where(df['IsHoliday'] == 1, df['Date'], pd.NaT)
    df['HolidayDate'] = pd.to_datetime(df['HolidayDate'])

    # Propagate historical and future horizons via forward/backward fill
    df['LastHoliday'] = df.groupby('Store')['HolidayDate'].ffill()
    df['NextHoliday'] = df.groupby('Store')['HolidayDate'].bfill()

    # Compute absolute chronological interval spans
    df['Days_Since_Holiday'] = (df['Date'] - df['LastHoliday']).dt.days
    df['Days_Until_Holiday'] = (df['NextHoliday'] - df['Date']).dt.days

    # Handle perimeter boundaries with extreme sentinel values
    df['Days_Since_Holiday'] = df['Days_Since_Holiday'].fillna(999).astype(int)
    df['Days_Until_Holiday'] = df['Days_Until_Holiday'].fillna(999).astype(int)

    # Purge intermediate analytical references
    df = df.drop(columns=['HolidayDate', 'LastHoliday', 'NextHoliday'])
```

```

return df

df = calculate_holiday_distances(df)

```

Tree Model Optimization Note: Numerical arrays generated via this method yield optimal conditional decision splits. For instance, the model can rapidly segregate rules matching `Days_Until_Holiday <= 2` to allocate distinct weights to pre-holiday volume spikes.

Approach 2: Discrete Window Indicator Flags (One-Hot Encoding)

When the target retail ecosystem experiences localized, highly temporal shockwaves immediately preceding or trailing a holiday, continuous numbers can sometimes introduce unwanted linear noise. Isolating explicit dates within a structured critical window via boolean indicators offers pristine categorical boundaries.

```

# Leverage pre-computed distance variables to build dedicated binary triggers
df['Is_Day_After_Holiday'] = np.where(df['Days_Since_Holiday'] == 1, 1, 0)
df['Is_2_Days_After_Holiday'] = np.where(df['Days_Since_Holiday'] == 2, 1, 0)

df['Is_Day_Before_Holiday'] = np.where(df['Days_Until_Holiday'] == 1, 1, 0)
df['Is_2_Days_Before_Holiday'] = np.where(df['Days_Until_Holiday'] == 2, 1, 0)

```

Approach 3: Non-Linear Gaussian Radial Basis Function (RBF) Transitions

This is an advanced feature engineering strategy. If continuous scales introduce far-horizon decay anomalies (e.g., a holiday being 45 days away skewing mathematical operations), we can subject the distance metric to a smooth Gaussian transformation. This map yields a peak value of exactly **1.0** at zero distance, which then gracefully decays exponentially toward **0.0** as time progresses.

$$\text{Holiday Effect} = \exp(-(\text{Days Distance})^2 / (2\sigma^2))$$

Where the tuning parameter σ (sigma) defines the geometric width or duration of the holiday's structural footprint (e.g., $\sigma = 3$ structures an influence scope wrapping roughly three days before or after the event).

```

# Smooth distribution wave formulation
sigma = 3 # Defines a multi-day decaying envelope of impact

df['Pre_Holiday_Wave'] = np.exp(-(df['Days_Until_Holiday']**2) / (2 * sigma**2))
df['Post_Holiday_Wave'] = np.exp(-(df['Days_Since_Holiday']**2) / (2 * sigma**2))

```

This provides neural networks or linear frameworks with a highly normalized, bounded scale ranging from **0.0** to **1.0**, signaling the precise mathematical exposure of that day to holiday traffic dynamics.